hw1

October 23, 2025

1 ChatGPT questions

Put this question ChatGPT and get the answer. You are allowed to discuss further. Please put a link to the response and your conversation. Given the response, you must first say whether: (i) the response is correct, (ii) the response is false, or (iii) you are really not sure. In the first case, give a succinct explanation of the proof. In the second case, explain why it is false. In the third case, explain your confusion. You do not need to write too much.

Conversations can be found here: here

1. Give a bijection from $\{x|x > 1, x \in \mathbb{R}\}$ and the interval $(0, 1)$.

   response

   the response is correct. it shows that for $x > 1, x \in \mathbb{R}$, the map $f(x) = \frac{1}{x-1}$ is injective and surjective to the interval $(0, 1)$.

1. NAE-3SAT is the language of 3-CNFs with an assignment that does not make all literals in a clause have the same value. Prove that NAE-3SAT is $\mathbb{NP}$-complete.

   response

   the proof is incorrect. the mapping from 3sat to nae3sat is invalid since the clauses are not 3 literals, and it doesnt give a way to convert the 4 literals into 3 literals.

2. Prove that if FACTORING is co-$\mathbb{NP}$-complete, then $TAUT \leq_{p(n)} SAT$ (recall that $TAUT$ is the language of tautologically true Boolean formulas.)

   response

   i am not sure if this proof is correct. it is almost certainly wrong but the words dont make any sense. claude is totally hallucinating.

2 regular problems

1. The Cantor-Schröder-Bernstein theorem states that if $|A| \leq |B|$ and $|B| \leq |A|$, then $|A| = |B|$. This is a fundamental theorem that shows that the infinities form an ordering.

   We will prove this theorem in this exercise. By assumption, there is an injection $f : A \to B$ and an injection $g : B \to A$. Construct a colored bipartite graph between A and B as follows. For each pair $(a, f(a))$, insert the red edge $(a, f(a))$. For each pair $(b, g(b))$, insert the blue edge $(g(b), b)$. (Note that $g(b) \in A$, so we flipped the order to describe the edge.) An *alternating path* is a finite path of edges that keeps switching colors: so either red, blue, red, blue, ..., or blue, red, blue, red, ...

   (a) Definite relation $a \to b$ if there is an alternating path from a to b. Prove that this relation is an equivalence relation.

   solution to prove equivalence relation, we need to show reflexivity, transitivity and symmetric.

   i. reflexivity:
   if a is connected to b through path c, then b is connected to a through c^{-1}.

   ii. transitivity:
   if a is connected to b with path y, and b to c through path z, then a must be connected through path $z \circ y$

   iii. symmetric:
   a is connected to itself with the trivial path

   (b) Prove that $A \cup B$ can be partitioned into set of: (i) alternating cycles, (ii) finite alternating paths, (iii) semi-infinite alternating paths, or (iv) doubly infinite alternating paths. (A semi-infinite alternating path has one endpoint; a doubly infininte alternating path has no endpoint.)

   solution

   Lemma 2.1. *if a connected component has a loop then the size of the loop must be the size of the connected component.*

   Proof. for a loop, there must exist some x ST $(fg)^n(x) = x$. if the connected component has some y such that $y \neq (fg)^n(y)$, there must be some $z = (fg)^m(y)$ such that $(fg)^n(z) = z$, $(fg)^{n-1}(z) \neq y$. but f is injective, so the existence of two distinct elements, y and $(fg)^{n-1}(z)$ mapping to z is a contradiction. □

   Lemma 2.2. *any finite subset of nodes must have either 0 or 2 terminal nodes.*

   Proof. a loop has 0 terminal nodes. a finite line has 2 terminal nodes. any other number of terminal nodes must have a junction similar to

lemma 2.1. if there is 1 terminal node, there must be a loop which is a contradiction by lemma 2.1. for any number of terminal nodes ≥ 3, there exists a subset of 3 elements such that the map of either f, or g is not injective. the three terminal nodes must meet at a junction node where the three map from 2 elements to one by the pigeonhole principle which is a contradiction. $\square$

paths have either 0 terminating points, 1 terminating point, or 2 terminating point. let us go over each case.

i. 0 terminating points:
if the connecetd component is finite, then it must loop. if the component has n points, then after $n+1$ jumps, it must have returned to a point it has already visited by pigeonhole. namely, it must return to the first element since if nothing returns to the first element, tracing the path backwards the first element would be an endpoint violating our assumption. (we will see a loop with an endpoint is impossible later)
if the component is infinite, we get the doubly infinite path. with no loops by lemma 2.1

ii. 1 terminating point:
there cannot exist a finite path with 1 terminating point by lemma 2.2
an infinite path with 1 terminating point must not have any loops, by lemma 2.2

iii. 2 terminating points:
if the path is finite, it must not have any loops
there cannot exist an infinite path with two terminating points. if there are two terminating points, there exists a finite subset of the path that has 3 terminating points which is a contradiction by lemma 2.2

(c) Construct a bijection between A and B, going over the four cases above.

solution

if there is a loop, either f or g is a bijection. if the graph is doubly infinite, either f or g is a bijection. if there is a singly infinite graph and the junction ends on $x \in A$, then f or f^{-1} on that component is a bijection. similarly for $y \in B$, g or g^{-1} is the bijection. for a finite non-loop, either f or g is a bijection.

2. Prove that the language

$\mathcal{L} = \{\langle M, x, 1^t\rangle | \text{M is a non-determinisitic TM that runs in t steps and accepts } x\}$ is $\mathbb{NP}$-complete.

solution

we will map SAT onto this language, and we will provide a certificate that runs in poly time.

there exists a way to encode boolean sat problems in unary. convert sat into binary , then conver the binary into unary. M_{sat}, the normal sat solver, will take these unary strings and outputs accept if satisfiable. this means that SAT is reducable to a subproblem in $\mathcal{L}$.

a certificate exists by just running M on a proposed string. since the machine M runs in polynomial time to the size of the input, and the length of the input is O(the size of the input), M will verify the input in polynomial time.

therefore the language is $\mathbb{NP}$-complete

3. formally prove that integer linear programming is $\mathbb{NP}$-complete.

   solution

   an integer program can be verified by plugging in the values into the constraints and checking if they are valid. computing Ax can be done in poly time, thus checking that constraints are satisfied thus it is in $\mathbb{NP}$.

   we prove that integer linear programming is np hard by mapping SAT onto it. all literals are either 0 or 1. for each clause, add a constraint saying each $\sum_{x_i \in c_n} f(x_i) > 0$ where

$$f : x_i \mapsto \begin{cases} x_i & x_i \text{if true} \\ (1 - x_i) & x_i \text{ is false} \end{cases} \tag{1}$$

   . there exists a solution of x_i iff the SAT problem is satisfiable because each corresponding clause in SAT is true. each clause is satisfied iff at least one var is true, iff each constraint is satisfied.

   if one of the literals is satisfied in the SAT program for cclause c_i, then in the correspoinding f_i, at least one x_j term is greater than 1.

   for each constraint, if the value is greater than 1, then one of the x_i terms is greater than 1, and finding this term gives the boolean in the SAT term that makes the clause true.